

Mario Tennis Fever Explorer

リポジトリ解説資料

この資料は、「自分が後から読み返して理解できること」と「事情を知らない他人に全体像を説明できること」を両立するための、学習用・引き継ぎ用のリポジトリ説明書です。

一言でいうと

HTML/CSS/JavaScript を中心に作られた、**静的な攻略・参照サイト**です。JavaScript が画面の表示と操作を担い、Node.js のスクリプトがページ生成やチェックを補助しています。

まず見るファイル

data.js: データ本体

js/main.js: 画面表示と操作

scripts/prerender.mjs: 英語ページ生成

この資料の読み方

まず「全体像」を掴み、そのあと「どの修正をどのファイルでやるか」を判断できるようになる構成にしています。

結論: React や Next.js のような大きいフレームワークではなく、素の JavaScript に近い形で組まれた静的サイトです。そのため「HTML が骨格」「CSS が見た目」「JS が動き」「Node.js スクリプトが生成補助」という分担で読むと理解しやすいです。

このリポジトリは何か

FAQ

初心者向けの質問、Tier の見方、負けたときの振り返り方をまとめる入口です。

Character / Racket / Court

各要素を比較しやすく一覧化する、データベース的な中核ページです。

Tips

試合で使う知識や操作メモを、検索・絞り込みしながら読めるようにしています。

Tier

自分基準の評価表をブラウザ内で編集し、画像として保存・共有できます。

役割

ゲーム攻略の「資料置き場」であり、ブログよりも参照性を優先したサイト。



性格

文章中心ではなく、検索・比較・一覧表示ができることが重要。



設計の結果

データを 1 か所に集め、同じ HTML を元に複数の静的ページを出力する構成になっています。

初心者向け一言まとめ: これは「アプリっぽく動く静的サイト」です。サーバーサイドで複雑な処理をしているわけではなく、ブラウザで動く JavaScript がかなり多くの役割を担っています。

使われている技術

技術	このリポジトリでの役割	初心者向けの見方
HTML	ページの骨格、各セクション、フォーム、カードを置く器	「どんな部品があるか」を定義する土台
CSS	配色、レイアウト、カード、モーダル、レスポンシブ表示	「見た目」と「配置」を決める層
JavaScript	翻訳、絞り込み、ソート、描画、保存、画像出力、初期化	「何が起きるか」を決める層
Node.js	英語ページの事前生成、英語フォールバックの検査、PDF出力補助	公開前にファイルを整える裏方
GitHub Actions	push / pull request 時に英語ページの品質チェックを実行	壊れていないかを自動で見る仕組み

この構成の特徴

- フレームワーク依存が薄く、ファイルの役割が直接的
- ビルドの責務が小さく、静的ホスティングしやすい
- データと UI の関係が追やすい

読むときの注意

- `main.js` は大きい 1 ファイルなので、上から全部読むと疲れやすい
- 処理順ではなく「翻訳」「フィルタ」「Tier」など役割単位で読む方が楽
- まずは関数名を見るだけでも構造がかなり分かる

初心者向け一言まとめ: このリポジトリは「難しいフレームワークを覚えないと読めない」タイプではありません。その代わり、JavaScript が 1 ファイルに多く集まっているので、役割で分けて理解するのがコツです。

フォルダとファイルの役割

```

/
├ index.html           ... 元になるHTML
├ data.js              ... キャラ / ラケット / コート / Tips / Tier関連データ
├ css/
│   └ style.css        ... サイト全体の見た目
├ js/
│   └ main.js          ... 画面表示・検索・保存・翻訳・初期化
├ scripts/
│   ├── prerender.mjs   ... 英語ページと各ルートHTMLを生成
│   ├── check-en-fallbacks.mjs ... 英語ページに日本語が残っていないか検査
│   └ export-repository-guide-pdf.mjs ... この資料のPDF出力
├ en/                  ... 生成済み英語ページ
├ faq/ characters/ rackets/ courts/ techniques/ tier/
│   └                  ... 生成済みの各ルートHTML
├ assets/              ... 画像素材
├ robots.txt / sitemap.xml ... 公開サイト向け補助ファイル
└ .github/workflows/    ... 自動チェック設定

```

場所	役割	見に行くタイミング
data.js	サイトに載せる実データを持つ	内容を直したいとき
js/main.js	検索、ソート、描画、保存、翻訳など画面の動作を持つ	挙動を直したいとき
css/style.css	見た目とレイアウトを持つ	デザインを直したいとき
scripts/	公開前の生成・検査を行う	ビルドや運用を理解したいとき

まず読む順 1

data.js を見て、どんなデータ型があるか把握する。

まず読む順 2

main.js の関数名一覧を見て、何をしているファイルかを掴む。

まず読む順 3

prerender.mjs を見て、生成済みページの意味を理解する。

初心者向け一言まとめ: 「どのファイルに触れば何がかわるか」を先に覚えると、コードを全部読まなくてもかなり動けます。

ページがどう作られているか

1. index.html

共通の土台となるページ骨格を持つ

→

2. js/main.js

各セクションの中身を描画し、操作に反応する

→

3. scripts/prerender.mjs

ルート別HTMLと英語ページを事前生成する

ブラウザで起きること

- main.js が data.js の配列を読み込む
- 現在の言語に応じて文言を切り替える
- 検索条件や並び順に応じてカードを再描画する
- お気に入りや Tier を localStorage に保存する

公開前に起きること

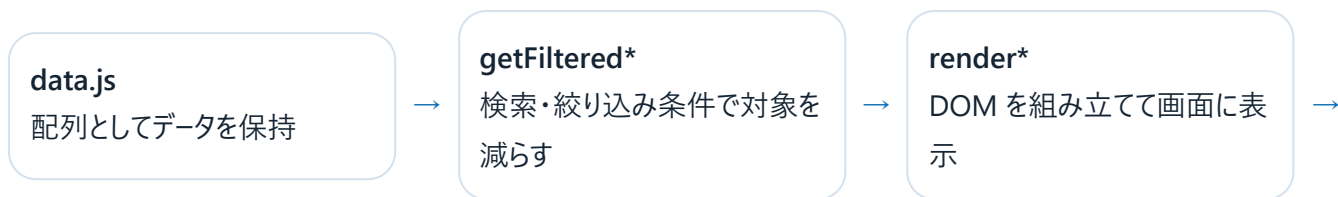
- prerender.mjs が各ルート用の HTML を書き出す
- SEO タグや英語用リンクを埋め込む
- check-en-fallbacks.mjs が英語ページの残留日本語を検査する
- GitHub Actions がこのチェックを自動実行する

生成済みディレクトリの意味

/faq/ や /characters/ などのディレクトリは、別プロジェクトではなく **同じテンプレートから出力された静的 HTML** です。つまり「ページが多い」のではなく「同じサイトをルート別に出力している」と考えると整理しやすいです。

初心者向け一言まとめ: 実際に編集する中心は少数のファイルです。見えている HTML ディレクトリが多くても、元をたどると「土台HTML + JS + データ + 生成スクリプト」に集約されます。

データの流れ



ユーザー操作

条件変更で再度描画

やりたいこと	主に触る場所	理由
キャラ説明文を直す	data.js	表示元のテキストがそこにあるから
検索条件を増やす	js/main.js	フィルタ処理と描画処理がそこにあるから
カードの見た目を直す	css/style.css	UIの装飾はCSSが担当しているから
英語ページ出力を変える	scripts/prerender.mjs	生成時の差し替え処理がそこにあるから

重要な発想

このサイトは「データを画面に流し込む」構成です。だから、まずデータがどう書かれているかを見ると、UIの意味も理解しやすくなります。

初心者が混乱しやすい点

HTMLに全ての内容が直書きされているわけではありません。実際にはJavaScriptがカードを作って挿入している部分が多いです。

初心者向け一言まとめ:「表示内容を直したいなら data.js」「動きを直したいなら main.js」「見た目を直したいなら style.css」と覚えると迷いにくいです。

JavaScript は何をしているか

1. 翻訳と言語切替

translations、t()、localizeValue() が中心。

日本語と英語の文言を持ち、現在の言語に合わせて取り出します。

2. フィルタと検索

getFilteredCharacters()、
getFilteredRackets()、getFilteredCourts()
が中心。

フォームの値を読み取り、配列を絞り込んでいます。

3. カード生成と描画

createCharacterCard()、
createRacketCard()、createCourtCard() と
render* 群が担当。

表示要素を組み立てて DOM に差し込みます。

4. 保存

loadFavoriteSet()、saveFavoriteSet()、
loadTierBoards()、saveTierBoards() が中心。

お気に入りや Tier 編集結果を localStorage に残
します。

5. 画像出力

buildTierBoardCanvas()、
exportTierBoardAsImage() が担当。

Tier 表を Canvas 化し、画像として保存・共有でき
るようにしています。

6. SEO と構造化データ

buildFaqStructuredData()、
updateStructuredData() が中心。

FAQ や一覧情報を検索エンジン向けの構造化デー
タとして出力します。

初期化の流れ

applyStaticTranslations()

固定文言を現在言語に合
わせる



renderCharacters /
Rackets / Courts / Tips
各一覧を描画する



renderAllTierBoards()
Tier 関連 UI を初期化する



updateStructuredData()

SEO 向け情報も更新する

main.js の読み方

1. 関数名だけざっと見る
2. getFiltered* と render* の対応を見る
3. 保存系と Tier 系を分けて読む

4. 最後に applyLocale() とイベント登録を見る

初心者向け一言まとめ: main.js を全部理解する必要はありません。「入力を受ける」「表示を作る」「保存する」の 3 種類に分けるだけで、かなり読めるようになります。

CHAPTER 7

英語対応とビルド

翻訳データ

main.js 内の translations
に文言を保持



prerender

ルート別HTMLと /en 以下の
英語ページを生成



check

英語ページに日本語フォールバ
ックが残っていないか確認

要素	何をしているか
scripts/prerender.mjs	各ルートの HTML を生成し、SEO タグ、alternate、英語リンクなどを差し込む
scripts/check-en-fallbacks.mjs	英語HTMLの中に日本語が残っていないか、必須の data-i18n 属性があるかを見る
.github/workflows/check-en-fallbacks.yml	push / pull request のたびにこのチェックを自動実行する

よく使うコマンド

```
npm run build
npm run check
npm run docs:repo-guide:pdf
```

初心者向け一言まとめ: このリポジトリの「ビルド」は複雑なJS変換ではなく、主に「静的ページを整えて書き出す作業」と「英語ページの品質確認」です。

このリポジトリで学べること

静的サイト構築

サーバーアプリを使わなくても、十分に実用的な参照サイトを作れる例です。

配列データ管理

コンテンツを配列で持ち、それを一覧 UI に流し込む考え方を学べます。

DOM操作

フレームワークなしでカードやモーダルを描画・更新する実装を読めます。

ローカライズ

1つのサイトを日本語/英語で使い分ける基本設計を学べます。

事前生成

静的サイトでもビルド時にページや SEO 情報を生成できることが分かります。

簡単な CI

GitHub Actions で、壊れやすいポイントを自動で確認する考え方を学べます。

JS 初心者ならこの順で読む

1. data.js でデータの形を見る
2. main.js の関数名一覧を見る
3. getFiltered* と render* を読む
4. お気に入り保存と Tier 保存を見る
5. prerender.mjs でビルドの意味を理解する

この資料を読んだあとにできるようになりたいこと

- このサイトが「データ・表示・生成」の 3 層でできていると説明できる
- 修正したい内容に応じて、どのファイルに触るべきか判断できる
- data.js と main.js の役割差を言える
- 英語ページ生成と自動チェックの流れを 1 段落で説明できる

初心者向け一言まとめ: このリポジトリは、JavaScript の実用コードを学ぶ題材としてかなり良いです。理由は、UI・データ・保存・多言語・ビルドが 1 つの題材にまとまっているからです。

要約メモ

質問	短い答え
これは何のリポジトリか	Mario Tennis Fever の攻略・参照用静的サイト
どの言語で作られているか	HTML / CSS / JavaScript、補助に Node.js
一番重要なファイルは何か	data.js、js/main.js、scripts/prerender.mjs
JS は主に何をしているか	翻訳、絞り込み、描画、保存、画像出力、初期化
ビルドは主に何をするか	静的HTML生成と英語ページの品質確認

Generated for local study and repository handoff.